A Vicuna-7B *vs.* LLaMA3-8B

At the start of our experimentation, we considered whether using a more recent and larger LLM might improve the performance of our LLM-based ASR setup. Although prior studies have shown that the combination of WavLM and Vicuna yields strong results (Ma et al. 2024; Yang et al. 2024; Kumar et al. 2025), we aimed to verify whether newer LLMs could offer further gains in our specific configuration.

To this end, we ran preliminary experiments across all selected datasets, using Prompt 1 (see Table 1) as the input prompt, and compared performance using Vicuna-7B[12] and LLaMA3-8B[13] as the decoder. Table 5 reports the resulting WER scores.

Since LLaMA 3 uses a distinct chat prompt formatting scheme, we adapted our template accordingly. The prompt was passed to LLaMA 3 as:

```
<|begin_of_text|>
<|start_header_id|>user<|end_header_id|>\n\n
Transcribe speech to text. {speech}<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>\n\n
```

Dataset	Word Error Rate	
	Vicuna (7B)	**LLaMA3 (8B)**
ContactCentre	11.91	**11.73**
CallHome	**25.26**	26.39
LS-TestOther	**5.59**	5.69
LS-TestClean	**2.88**	3.37
AMI	13.72	**13.33**

Table 5: Word Error Rates (WER) across datasets for Vicuna (7B) and LLaMA3 (8B)

The results showed that LLaMA3 outperformed Vicuna on only two of the five datasets (CC and AMI), with Vicuna achieving comparable or better performance on the others. Although LLaMA3 was pre-trained on a much larger corpus (15 trillion tokens) and emphasizes general-purpose capabilities such as reasoning, coding, and broad knowledge understanding, Vicuna was fine-tuned specifically for conversational alignment and instruction-following. This targeted specialization may account for its stronger performance in our ASR setup, potentially making it easier to align the projected speech embeddings with the LLM's input space.

B Impact of k in the LLM-based ASR + Prompt Tuning Experiments

As discussed in Section 5.3, we conducted a series of experiments using prompt tuning as an alternative soft-prompting technique. A critical hyperparameter in this approach is the number of learnable embeddings, k, which can significantly affect performance and is challenging to tune effectively.

[12]https://huggingface.co/lmsys/vicuna-7b-v1.5

[13]https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

To illustrate the sensitivity of prompt tuning and contrast it with the simplicity of our proposed Prompt Projector module, we performed additional experiments using prompt tuning with $k \in \{10, 20, 28, 30, 50, 75\}$. These experiments were carried out on the CC and CH datasets.

For these experiments, we selected two different prompts from those listed in Table 1. Specifically, Prompt 1 was used for the ContactCentre dataset, while the *base* prompt was chosen for the CallHome dataset. The results, illustrating the impact of the number of learnable embeddings, are presented in Figures 5 and 6, respectively. The bar labeled PP in each figure corresponds to the performance of our proposed Prompt Projector module, applied using the same prompt.

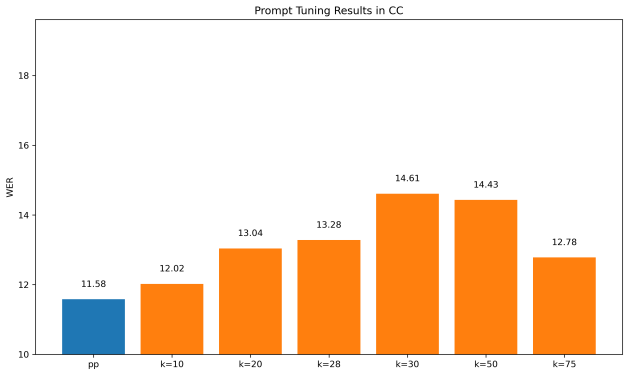

Figure 5: ASR performance (WER) across different values of k (number of learnable embeddings) using the prompt tuning approach in the ContactCentre dataset. For this set of experiments, we used Prompt 1 from Table 1 as the discrete initial prompt. The bar labeled $pp(\cdot)$ shows the performance of our proposed approach using the same prompt.

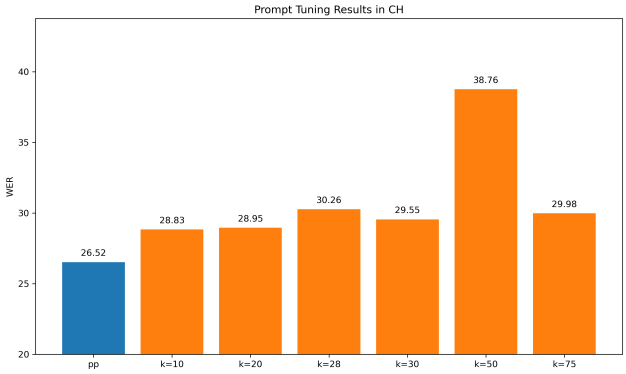

Figure 6: ASR performance (WER) across different values of k (number of learnable embeddings) using the prompt tuning approach in the CH dataset. For this set of experiments, we used Prompt *base* from Table 1 as the discrete initial prompt. The bar labeled $pp(\cdot)$ shows the performance of our proposed approach using the same prompt.

As observed in our experiments, varying the value of k leads to significant differences in WER, underscoring the

challenge of selecting an optimal number of learnable embeddings for prompt tuning. While a more comprehensive exploration is needed, these ablation results suggest that around $k = 10$ learnable embeddings may be the preferable choice for ASR tasks.

B.1 To Freeze, or Not to Freeze

In Section 3.2, we emphasized the importance of keeping the underlying model frozen when training the *Prompt Projector*. Here, we present a series of experiments to support this claim. Given a trained *vanilla* model with trained $sp(\cdot)$, we compare two scenarios: (1) **unfrozen** — the *vanilla* model continues updating $sp(\cdot)$ while training $pp(\cdot)$, and (2) **frozen** — only $pp(\cdot)$ is trained, keeping the *vanilla* model frozen including $sp(\cdot)$.

Figure 7 shows the WER progression over five training epochs for both scenarios across all prompts, using the *ContactCenter*, *CallHome*, and *AMI* datasets. When the *vanilla* model is not frozen (green boxplots) and continues training alongside $pp(\cdot)$, we observe significant performance instability, *i.e.*, WER is higher. This degradation is primarily due to interference and catastrophic forgetting in the speech projector $sp(\cdot)$. By contrast, freezing the model (purple boxplots) leads to much more stable training, allowing the optimization to focus exclusively on learning a better prompt projection, without altering the already learned "speech-to-LLM" projection.

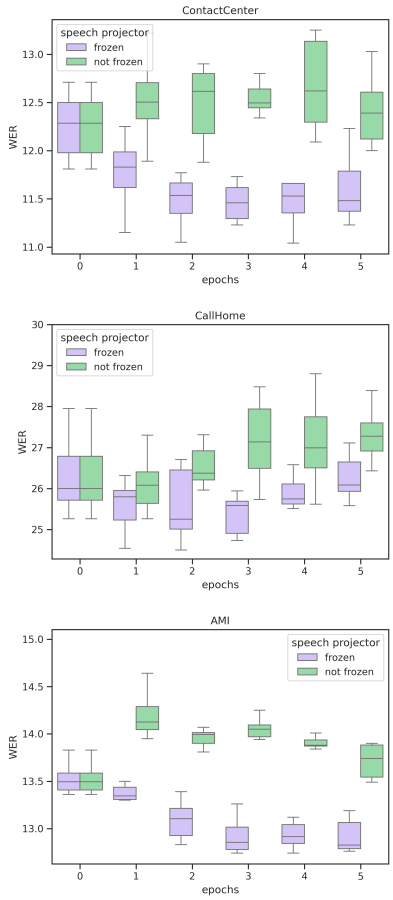

Figure 7: Impact of freezing *vs.* not freezing the base *vanilla* model while training the $pp(\cdot)$ module. The figure shows evaluation results at each epoch during training. Epoch 0 represents the performance of the already trained *vanilla* model, before starting the training of $pp(\cdot)$. The boxplots display the WER distributions for all the different prompts.